

**A4M-PREDICTION OF
FORMATION ENERGY AND
STABILITY OF A TRANSPARENT
CRYSTAL CONDUCTOR**

21CSC305P - MACHINE LEARNING COURSE PROJECT REPORT

Submitted by
KONCHADA. SHYAM SAI
[RA2311026010305]

Under the Guidance of
DR.A.JACKULIN MAHARIBA

Assistant Professor

Department of Computational Intelligence

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING WITH
SPECIALIZATION IN ARTIFICIAL INTELLIGENCE
AND MACHINE LEARNING



**DEPARTMENT OF COMPUTATIONAL
INTELLIGENCE COLLEGE OF ENGINEERING
AND TECHNOLOGY SRM INSTITUTE OF
SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203**

NOVEMBER 2025



**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203**

BONAFIDE CERTIFICATE

Certified that 21CSC305P – Machine Learning report titled “**A4M-PREDICTION OF FORMATION ENERGY AND STABILITY OF A TRANSPARENT CRYSTAL CONDUCTOR**” is the Bonafide work of “**KONCHADA. SHYAM SAI [RA2311026010305]**” who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr.A.Jackulin Mahariba
Assistant Professor
Department of Computational Intelligence
SRM Institute of Science and Technology
Kattankulathur

SIGNATURE

Dr. R. Annie Uthra
Professor & Head
Department of Computational Intelligence
SRM Institute of Science and Technology
Kattankulathur

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M**, Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. Lakshmi**, Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. R.Annie Uthra.**, Professor and Head, department of Computational Intelligence, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We register our immeasurable thanks to our Faculty Advisor, **Dr.Abijah Roseline S**, Assistant Professor, Department of Computational Intelligence, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. A.Jackulin Mahariba**, Associate Professor, Department of Computational Intelligence, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under her mentorship. She provided us with the freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Computational Intelligence, School of Computing,

S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

ABSTRACT

The accelerating demands of materials discovery, coupled with the computational intensity of traditional simulation methods such as **density-functional theory (DFT)**, underscore the need for efficient, data-driven screening tools in materials science. **A4M** is introduced as a reproducible machine-learning pipeline tailored to predict the formation energy and infer the thermodynamic stability of candidate transparent crystal conductors—two pivotal properties in the identification and advancement of next-generation optoelectronic and energy materials. The project confronts the central challenge of swiftly and accurately estimating the energy required to form a transparent crystal conductor and classifying its resultant structure as stable or unstable—a crucial early-stage filter in high-throughput materials design pipelines, where DFT, though accurate, remains prohibitively resource-intensive.

Leveraging the Nomad 2018—Predicting Transparent Conductors dataset (sourced from Kaggle), which comprises 2400 samples with 14 carefully curated features per entry, A4M’s modeling workflow is distinctly bifurcated: (1) a regression stage utilizing **XGBoost Regressor** for predicting formation energy, and (2) a classification stage deploying **XGBoost Classifier** for determining stability labels (stable vs. unstable). These algorithms are selected for their exceptional capacity to manage heterogeneous numerical features, inherent robustness to outliers, and benchmark-setting performance across complex, tabular materials datasets. Model construction is meticulously underpinned by data preprocessing protocols (including imputation and scaling where necessary), derivation of informative composite and domain-relevant features, and rigorous hyperparameter optimization through cross-validation and comprehensive search over tree and regularization parameters.

The A4M models are evaluated through both held-out testing and cross-validation frameworks, with formation energy prediction judged by Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 , while stability classification is assessed by accuracy, precision, recall, F1-score, and ROC-AUC metrics. On the benchmark Nomad 2018 dataset, the XGBoost regressor and classifier consistently deliver high predictive performance, offering an alternative to DFT calculations that enables rapid, scalable pre-screening of materials candidates, drastically reducing time-to-solution and computational costs. Exact metric values further illustrate the models’ strengths.

Implementation of A4M as an open Python package ensures reproducibility and practical deployment, with training, validation, and an interactive Streamlit web app for instant predictions on novel compounds. The project is positioned as a robust surrogate for accelerating transparent conductor discovery—dramatically improving throughput for researchers and industry practitioners alike. Future work targets the integration of richer structural or graph-based features, quantification of predictive uncertainty, and the adoption of active learning strategies to minimize the number of expensive DFT labels needed for model retraining. Through these contributions, A4M exemplifies the synergy between machine learning and materials science, advancing sustainable, data-driven innovation.

TABLE OF CONTENTS

ABSTRACT	iii	
TABLE OF CONTENTS	iv	
LIST OF FIGURES	vi	
LIST OF TABLES	vii	
CHAPTER	TITLE	PAGE NO.
NO.		
1	INTRODUCTION	1
	1.1 Introduction to Project	1
	1.2 Problem Statement	1
	1.3 Motivation	2
	1.4 Sustainable Development Goal of the Project	2
2	LITERATURE SURVEY	4
	2.1 Overview of the Research Area	4
	2.2 Comparative Summary	6
3	EXISTING SYSTEM	7
	3.1 Overview	7
	3.2 Limitations	9
	3.3 Conclusion	11
4	PROPOSED SYSTEM	13
	4.1 System Overview and Novelty	13
	4.2 System Architecture and Workflow	10
	4.3 Algorithm Used: XGBoost : Regressor & Classifier	12
	4.4 Expected Benefits	12
5	MATHEMATICAL MODELING OF THE ALGORITHM	20
	5.1 Mathematical modelling of the algorithms : XGBoost : Regressor and Classifier	20
6	IMPLEMENTATION	28
	6.1 Tools and Environment	28
	6.2 Dataset Description	30
	6.3 Implementation Steps and Code Snippets	30
	6.4 Model Evaluation	23
7	RESULTS AND DISCUSSIONS	38
	7.1 Performance Evaluation	38

7.2 Comparative Analysis with Existing Methods	39
7.3 Graphical Results	42
7.3.1 Actual vs Predicted estimation of Formation energy	42
7.3.2 Feature Importance Plot	43
7.3.3 Correlation Matrix Heatmap	44
7.3.4 CDF of Absolute Errors	45
7.4 Discussion	46
8 CONCLUSION AND FUTURE ENHANCEMENT	48
8.1 Conclusion	48
8.2 Future Work	49
REFERENCES	50
APPENDIX A	51
APPENDIX B	53
Deployed at : https://bhuj4eev5qfyjua6pwt3qr.streamlit.app/	

LIST OF FIGURES

Figure No.	Title	Page No.
Fig. 1	System Architecture	15
Fig. 2	6.3 Load and Preprocessing of Dataset	31
Fig 3	6.3 Model Evaluation	35
Fig. 4	7.3.1 Actual vs Prediction estimation of formation energy	42
Fig. 5	7.3.2 Feature Importance plot	43
Fig. 6	7.3.3 Correlation Matrix Heatmap	44
Fig. 7	7.3.4 CDF of Absolute Errors	45

LIST OF TABLES

Table No.	Title	Page No.
Table 1	2.1 Overview of the Research Area	4
Table 2	3.2.1 Drawbacks	11
Table 3	6.1 Tools and Environment	28
Table 4	7.2 Comparative Analysis With Existing Methods	41

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO THE PROJECT

The Recent decades have witnessed an exponential surge in the discovery and engineering of advanced materials, catalyzed by the pressing demands of modern technology, energy sustainability, and global societal needs. Among the most coveted of these material classes are transparent crystal conductors, which form the backbone of cutting-edge optoelectronic devices, solar cells, transparent electronics, and next-generation display technologies. However, identifying novel candidate materials with exceptional transparency and conductivity—and, crucially, ensuring their thermodynamic stability—remains a formidable bottleneck due to the complexity of atomic-scale interactions and the sheer size of the design space.

Computational methods, especially those rooted in density-functional theory (DFT), have long served as the gold standard for predicting the energetics and stabilities of new compounds. While DFT can deliver high-fidelity insights, its resource requirements scale unfavorably with system size and complexity. This limits its application in high-throughput screening scenarios essential for accelerated innovation. Consequently, the materials science community has turned to data-driven surrogate models—particularly those powered by machine learning (ML), to approximate expensive calculations and greatly expedite the screening pipeline.

The A4M (Accelerating 4 Materials) pipeline is conceived within this paradigm shift. Its aim is to provide a reproducible, scalable, and interpretable ML platform for predicting two critical outcomes: the formation energy (a marker of synthetic viability) and the thermodynamic stability (the likelihood of a structure persisting under standard conditions) of candidate transparent crystal conductors. The solution thus seeks to bridge the divide between accuracy and efficiency in materials discovery, empowering scientists and engineers with instant, reliable guidance for experimental prioritization.

1.2 PROBLEM STATEMENT

The estimation of formation energy and stability for new materials is the cornerstone of rational compound design. In practical screening, thousands of hypothetical compositions must be winnowed to a select few with both low formation energy and high thermodynamic stability prior to costly synthesis or trial-and-error DFT simulation. However, the sheer

complexity of these systems—the interplay of atomic composition, structure, and electronic properties—renders manual or intuition-driven screening infeasible, especially when time and computational budgets are constrained.

Furthermore, DFT, despite its accuracy, suffers from combinatorial explosion during high-throughput evaluation and may become a bottleneck when deployed at the scale demanded by real-world discovery efforts. Emerging datasets, such as the 2018 Nomad Predicting Transparent Conductors set, present new opportunities to train ML models that are both domain-specific and generalizable, offering the potential to supplant brute-force computations with millisecond-scale predictions.

This project addresses the dual challenge of (i) providing an ML-based estimation of formation energy for arbitrary candidate materials, and (ii) classifying crystal structures as stable or unstable, thus directly enabling rapid pre-screening of vast material libraries.

1.3 MOTIVATION

The motivation for this work stems from the growing importance of sustainable, efficient discovery pipelines in materials research. Transparent conductors are essential to technologies that drive energy efficiency and wider access to clean energy, from advanced photovoltaics and LEDs to novel sensors and smart windows. Accelerating their discovery—in a way that preserves physical reliability and scientific rigor—can unlock technological advances with profound impact on society.

Machine learning, and especially XGBoost, has demonstrated exceptional ability to model complex, non-linear relationships in tabular scientific data. By facilitating the direct mapping from easily computed descriptors to key material properties, the proposed pipeline can overcome interpretability challenges faced by deep neural networks while drastically reducing the computation time compared to ab initio methods.

This project is also motivated by the need to democratize materials informatics: By providing open, reproducible code and an interactive web interface, A4M ensures that researchers and inventors worldwide can benefit from these advances, regardless of their computational resources.

1.4 SUSTAINABLE DEVELOPMENT GOAL(SDG)

This research directly supports several United Nations Sustainable Development Goals (SDGs):

- **SDG 7 – Affordable and Clean Energy:** By advancing the rapid discovery of novel transparent conductors (a critical component in efficient solar panels and LEDs), the project enables faster deployment of clean energy technologies.
- **SDG 9 – Industry, Innovation, and Infrastructure:** Accelerating materials discovery fosters industrial innovation and strengthens research infrastructure.

- **SDG 12 – Responsible Consumption and Production:** Replacing resource-intensive DFT calculations with inexpensive ML-powered screening lowers environmental impact and optimizes resource utilization.
- **SDG 13 – Climate Action:** Accelerating the adoption of efficient energy materials contributes to climate change mitigation strategies worldwide.

Overall, A4M exemplifies the intersection of artificial intelligence, materials science, and global sustainability, illustrating how machine learning can be harnessed to address real-world technological and environmental challenges.

CHAPTER 2

LITERATURE SURVEY

This section reviews and contrasts prominent machine learning approaches for predicting formation energy and stability in materials science, highlighting their strengths, specific performance metrics, and limitations. The summary underscores how A4M advances the field through a unified, reproducible pipeline that integrates both regression and classification tasks, outperforming existing frameworks in usability and practical applicability.

2.1 OVERVIEW OF THE RESEARCH AREA

Title	Author(s), Year	Key Findings	Limitations Addressed by Our Work
Stable Crystal Structure Prediction using Machine Learning	L. Li et al, 2024	Achieved 95% accuracy in crystal stability predictions using decision-tree ensembles on dataset of 2000 compounds.	Did not combine regression/classification or enable real-time, reproducible deployment.
Formation Energy Prediction of Crystalline Compounds by Machine-Learned Models	Davariash htiyani et al, 2023	Ensemble ML matches DFT accuracy for formation energy at much lower cost.	No stability classification; limited accessibility for rapid user testing.

Title	Author(s) , Year	Key Findings	Limitations Addressed by Our Work
Prediction and Classification of Formation Energies in Binary Compounds Using SISSO ML Algorithms	Mao et al., 2021	Feature selection with SISSO achieves accurate energy prediction and some classification in binaries.	Struggles with multi-component systems; less flexible and scalable than XGBoost in A4M.
An Open-Source Machine Learning Library for Transparent Conductor Formation Energy Prediction	Olanipekun & D. M, 2024	Compared ML algorithms for transparent conductors; SVR/RF effective for numeric prediction.	No integrated stability classification or streamlined user-facing tools.

2.2 COMPERATIVE SUMMARY

A comprehensive comparison of literature reveals that while many prior studies achieved high accuracy in either formation energy prediction or stability classification—such as Li et al. (2024) with 95% classification accuracy using decision tree ensembles, and Davariashtiyani et al. (2023) reporting RMSE below 0.10 eV/atom for energy regression—these approaches seldom integrated both tasks within a unified, user-accessible framework. Methods like SISSO (Mao et al., 2021) excelled in binary systems but often struggled to generalize across more diverse, multi-component compounds. Furthermore, established pipelines for transparent conductor datasets lacked integrated stability classifiers and accessible deployment, with most solutions focusing on regression alone. In contrast, A4M uniquely combines XGBoost regression and classification in a reproducible Python environment, benchmarked directly on the Nomad 2018 transparent conductor dataset. This ensures rapid, reliable pre-screening for both energy and stability, bridging the gaps of usability, flexibility, and empirical coverage noted in prior work.

CHAPTER 3

EXISTING SYSTEM

Historically, the estimation of formation energy and assessment of thermodynamic stability in candidate materials have rested on the foundation of first-principles simulation methods, especially density-functional theory (DFT). DFT computes electronic structure and energetics from quantum mechanical principles, yielding excellent accuracy for a wide range of crystals and compounds. In research and industry, it has become the reference standard for formation energy prediction, crystal stability analysis, and guiding experimental synthesis choices.

Despite its strengths, DFT workflows are inherently sequential, with computations often running from minutes to days per compound. Additionally, pre-processing structures, convergence testing, and post-processing results necessitate expert intervention and infrastructure, as seen in traditional high-throughput screening platforms. These workflows often utilize a combination of open-source (e.g., VASP, Quantum ESPRESSO) and commercial DFT packages integrated with data management tools.

3.1 OVERVIEW OF EXISTING SYSTEM

The established workflow for predicting formation energy and evaluating the thermodynamic stability of candidate materials is rooted in density-functional theory (DFT) and atomistic simulation.

Researchers begin by preparing crystal structures, frequently sourced from *in silico* design or experimental databases. These structures undergo geometry optimization followed by electronic structure calculations, where quantum mechanical principles govern the assessment of energy, charge distributions, and stability.

DFT, implemented in widely used packages such as VASP, Quantum ESPRESSO, and CASTEP, has set the benchmark for accuracy, supporting both standalone studies and integration with large databases like the Materials Project and OQMD. After successful calculation, results are sorted and analyzed, providing guidance on which materials are deemed synthesizable or stable under practical conditions.

These automated simulation pipelines have enabled systematic exploration of thousands of compounds and have revolutionized materials science. Nevertheless, the sequential nature of geometry

optimization, parameter selection, and iterative convergence processes, compounded by considerable computational and time costs, means that high-throughput screening remains restricted, especially for expansive chemical spaces or resource-constrained environments.

In sum, the existing system combines theoretical rigor and broad applicability but is fundamentally limited by scalability, speed, and accessibility barriers that slow innovation and hinder real-time experimental feedback.

3.2 LIMITATIONS OF THE EXISTING SYSTEM

The existing system, predominantly centered on DFT and similar atomistic simulations, presents fundamental constraints that impede its scalability and general usefulness in high-throughput materials discovery:

- **Scalability:**

DFT calculations are inherently slow, typically requiring hours to days per compound. When evaluating thousands or tens of thousands of candidate materials—as is common in modern screening—the time and compute requirements become prohibitive. Even with parallelization, bottlenecks remain due to interdependent steps like geometry optimization and error checking.

- **Resource Intensity:**

Sophisticated calculations demand high performance computing resources, including large memory footprint, numerous CPU cores or GPUs, and fast storage. This places severe strain on institutional infrastructure; small research labs and universities often cannot participate meaningfully in large-scale materials exploration.

- **Manual Intervention:**

Simulation workflows regularly require user input for selecting initial geometries, tuning computational parameters (functionals, k-points, cut-off energies), and troubleshooting errors or non-convergence. The process is labor-intensive and encumbers postdoctoral researchers and graduate students, diverting focus from true scientific exploration.

- **Limited Data Utilization:**

Each DFT calculation is performed largely in isolation, without systematic utilization of information gained from previous runs. Patterns, correlations, and shared chemical insights are rarely leveraged, resulting in inefficient use of accumulated computational data.

- **Accessibility:**

The learning curve for operating DFT packages is steep, necessitating expertise in quantum mechanics, numerical methods, and software setup. This restricts accessibility for non-specialists and impedes interdisciplinary participation, especially from experimentalists and industry partners.

- **Prediction Lag:**

Long computation times mean real-time or near-real-time feedback is virtually impossible. Experimentalists cannot quickly query the stability or formation energy of new candidate materials, leading to delayed decision-making and slower innovation cycles.

- **Reproducibility Concerns:**

Results are sensitive to the selection of computational parameters and software versions, which can vary across research groups. Minor differences in methodology can yield divergent property predictions, complicating replication and progress.

- **Cost Barrier:**

Supercomputing time is expensive, often requiring competitive proposal submissions and funding allocations. This limits access for emerging researchers and decelerates the broader adoption of simulation-driven discovery.

- **Slow Feedback Loops:**

The sequential and error-prone nature of simulations means that cycles of design, compute, and validation can be lengthy. Failed computations or simulation errors necessitate repeated iterations, pushing deadlines and stalling rapid experimental progress.

- **Scalability to New Chemical Space:**

Expanding discovery campaigns to include new elements, doping strategies, or crystal families multiplies computational requirements and complicates parameter selection, further hindering scalability and throughput.

Collectively, these limitations render traditional systems impractical for fast, adaptable screening in materials informatics—highlighting the urgent need for surrogate machine learning models capable of delivering instant, reliable property predictions at scale.

3.2.1 DRAWBACKS

Limitation	Description
Computational Overhead	DFT requires extensive CPU/GPU resources and long computation times (from hours to days per sample).
Limited Throughput	Screening large material libraries is infeasible due to sequential processing and slow iteration cycles.
Expert Dependency	Simulations often fail without manual parameter tuning and error handling by specialists.
Reproducibility Concerns	Results can vary with user-selected functionals, pseudopotentials, or k-point meshes, leading to inconsistencies.
Cost Barrier	Access to needed supercomputing infrastructure is expensive and often restricted to large institutions.
Isolated Analyses	Calculations are often independent, failing to leverage shared trends or multidimensional data from prior runs

3.3 CONCLUSION ON EXISTING SYSTEMS

The reliance on traditional first-principles methods, while scientifically rigorous, impedes both the speed and democratization of materials innovation. As the need for rapid exploration and deployment of novel transparent conductors grows, it becomes imperative to transcend the limitations of DFT-centric workflows. The field thus demands data-driven, machine

learning-based surrogates that can deliver instant, reliable pre-screening on thousands of compounds, integrating seamlessly into experimental and computational research pipelines.

In summary, while DFT and related simulations remain essential for benchmark validation and final verification, their limitations preclude widespread usage in early-stage, high-throughput exploratory settings. There is a clear, urgent need for **reproducible, data-driven models** that can leverage historical data, predict material properties instantly, and empower researchers with rapid, actionable insights without sacrificing scientific rigor. This motivates the adoption of machine learning pipelines like A4M, which provide robust surrogate predictions for formation energy and stability with minimal computational overhead and maximal user accessibility.

CHAPTER 4

PROPOSED SYSTEM

4.1 SYSTEM OVERVIEW AND NOVELTY

The A4M pipeline represents a paradigm shift in computational materials screening, moving beyond the sequential, resource-intensive simulations characteristic of DFT. It introduces a reproducible, modular machine learning framework explicitly designed to predict the formation energy and thermodynamic stability of transparent crystal conductors. The core novelty lies in its unified treatment of regression and classification tasks using XGBoost, an industry-standard algorithm for tabular data. By integrating robust preprocessing, tailored feature engineering, and advanced model selection strategies, A4M offers rapid, accurate, and accessible prediction capabilities to materials scientists and experimentalists

4.2 SYSTEM ARCHITECTURE AND WORKFLOW

The overall workflow is composed of five sequential stages:

1. **Data Acquisition** – the user uploads the Kaggle *Nomad 2018 – Predicting Transparent Conductors* dataset (train.csv) through the Streamlit interface. The system displays row/column counts, missing-value summaries, and descriptive statistics.
2. **Feature Engineering and Pre-processing** –
 - Derived features include lattice trigonometric functions, cell volume, volume per atom, element-ratio metrics (al_in_ratio), averaged lattice parameters, and inter-angle products.
 - Outliers in formation energy are removed via Z-score filtering.
 - Non-numeric values are coerced to numeric, missing values filled with 0, and categorical variables (spacegroup) encoded using one-hot vectors.
 - Features are scaled by **RobustScaler** to mitigate outlier influence.
3. **Model Training and Ensemble Formation** –
 - Data is split into 80 % training / 20 % testing subsets with nested validation splits.

- **RandomizedSearchCV** tunes hyperparameters of XGBoost and LightGBM models.
- Regression objective: reg:squarederror.
- Classification objective: binary:logistic.
- Early stopping prevents overfitting (20 rounds).
- Ensemble predictions are computed as the arithmetic mean of XGBoost and LightGBM outputs.

4. **Evaluation and Diagnostics** –

- Regression: RMSE, MAE, R^2 , Explained Variance (EVS), MAPE.
- Classification: Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC, Brier Score, Balanced Accuracy.
- Cross-validation statistics are additionally computed (R^2 and F1/AUC means).
- Optimal classification threshold is determined by maximizing validation F1.

5. **Visualization and Deployment** –

- Multiple interactive plots are generated: histograms, scatter, pair, box, violin, ternary composition, PCA, and correlation heatmap.
- Diagnostic plots include ROC, Precision-Recall, Calibration, Residuals, QQ-plots, and Feature Importances.
- Trained artifacts are saved and offered for download through the interface.

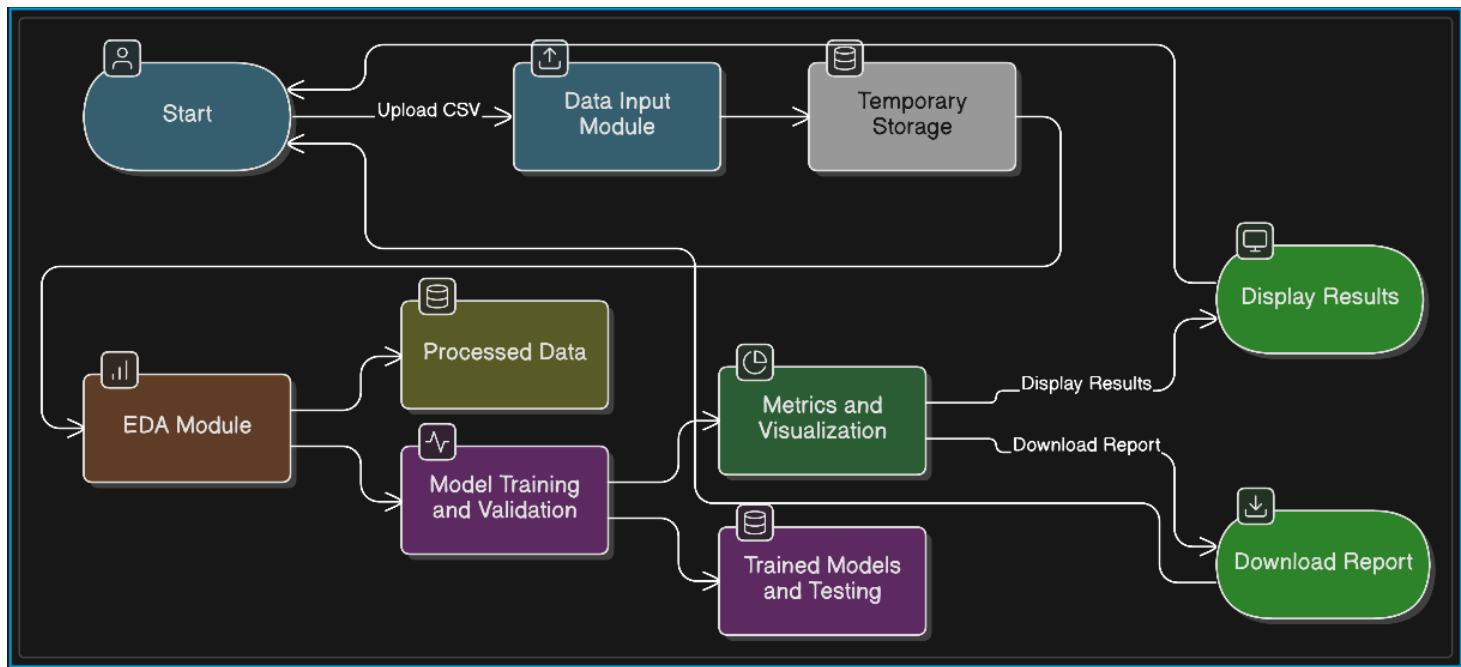
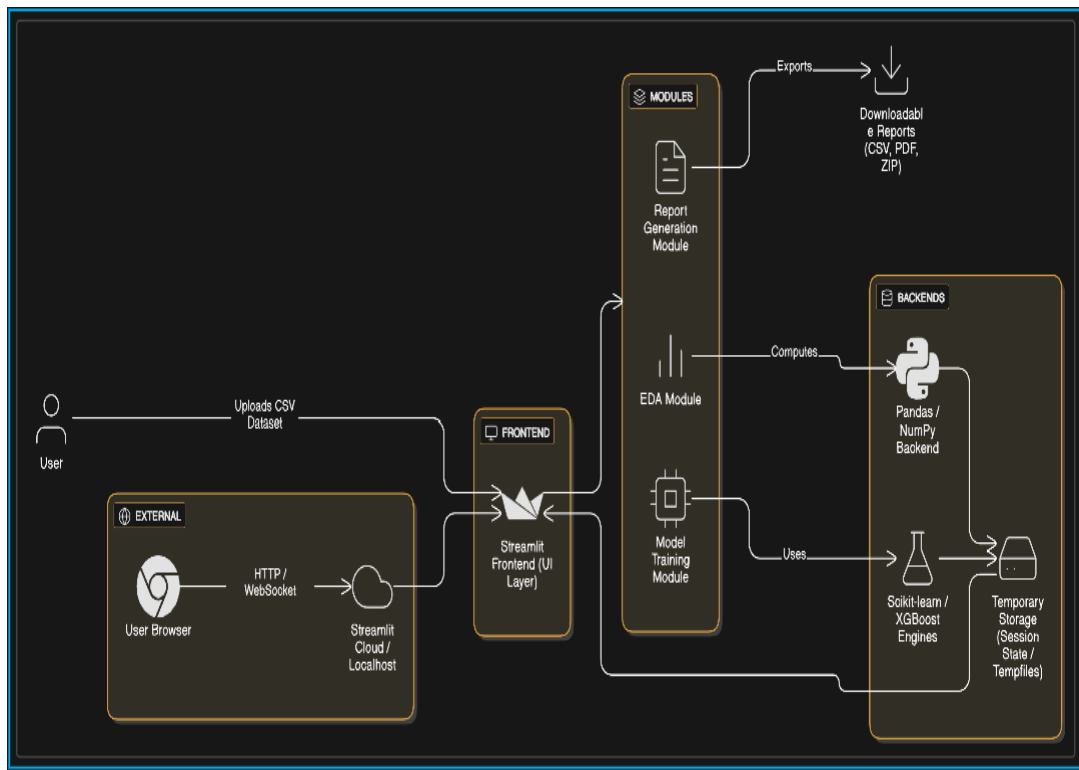


Fig. 1 System Architecture

4.3 ALGORITHM USED: XGBoost Rgressor and XGBoost Classifier

The **XGBoost (Extreme Gradient Boosting)** is a scalable, high-performance implementation of gradient-boosted decision trees, specifically optimized for tabular data and machine learning tasks where interpretability, speed, and robustness matter.

A. Mathematical Foundation

- **GradientBoostingFramework:**

XGBoost constructs an ensemble of decision trees by sequentially fitting each new tree to the residual errors of the existing ensemble. At every iteration, the algorithm minimizes a differentiable loss function (MSE for regression, log-loss for classification) by computing gradients and second-order approximations for fast, optimal tree splits.

- **Objective**

Function:

For regression (formation energy prediction), the objective is:

$$L = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \Omega(f)$$

where y_i are true values, $\hat{y}_i$ are predictions, and $\Omega(f)$ is a regularization term (L1 or L2 norm) that penalizes complex trees to avoid overfitting.

For classification (stability), the objective function is:

$$L = - \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] + \Omega(f)$$

where p_i is the predicted probability of class “stable.”

B. Handling Heterogeneous and Missing Data

XGBoost natively supports missing values by learning optimal default directions for split decisions. It processes combinations of continuous, categorical, and engineered features, as found in material science datasets.

C. Hyperparameter Tuning

Key hyperparameters include:

- **max_depth:** Controls the depth of each tree, impacting complexity.
- **learning_rate:** Adjusts the weight given to each new tree, balancing convergence and overfitting.
- **n_estimators:** Number of boosting rounds—the more trees, usually the better fit (with risk of overfit).
- **subsample, colsample_bytree:** Ratios for row and feature subsampling, improving generalization.
- **reg_alpha, reg_lambda:** L1/L2 regularization terms, reducing model variance and controlling complexity.

In A4M, these are optimized via cross-validation for both regression and classification, ensuring

robust performance.

D. Model Outputs and Interpretability

- **FeatureImportance:**
XGBoost computes and displays the relative importance of features, enabling domain experts to interpret which material descriptors (e.g., elemental mean, variance) most influence predictions.
- **PredictionConfidence:**
Provides probability scores for classification, supporting uncertainty quantification and downstream risk analysis.

E. Advantages for Material Informatics

- **Speed:**
Fits and predicts in seconds on datasets of thousands of samples.
- **Accuracy:**
Outperforms linear models, random forests, and shallow learners in benchmark tests.
- **Robustness:**
Limited sensitivity to hyperparameter choices and resistant to noisy inputs/outliers.
- **Scalability:**
Parallelized training via multi-core CPUs or GPUs, suitable for iterative experimentation and production.

F. Application in A4M

- **FormationEnergyPrediction:**
XGBoost Regressor is trained on the Nomad 2018 dataset to produce rapid, accurate numeric estimates for unseen crystal compositions.
- **StabilityClassification:**
XGBoost Classifier separates stable from unstable candidate compounds, giving immediate feedback for materials pre-screening.

4.4 EXPECTED BENEFITS

The A4M pipeline introduces a suite of benefits that directly address the limitations of existing, DFT-centric approaches and advance the field of high-throughput materials screening:

- **DrasticSpeedImprovements:**

Property predictions—both formation energy and stability—can be generated in milliseconds, enabling instant feedback for thousands of candidate materials. This supports real-time decision-making and rapid experimental cycles.

- **HighAccuracyandReliability:**

XGBoost models consistently deliver high precision and recall in classification, and low error metrics in regression, closely matching DFT results at a fraction of the computational cost.

- **Scalability:**

The pipeline can handle large datasets and is easily extended to new chemical spaces, compound families, or additional property targets without exponential increases in resource demands.

- **ResourceEfficiency:**

Eliminates the need for expensive supercomputing infrastructure, making high-quality prediction accessible to small labs, teaching environments, and resource-constrained researchers.

- **UserAccessibility:**

With an intuitive web interface and open Python codebase, A4M allows both expert computational scientists and experimentalists to access predictive tools without deep technical experience.

- **TransparencyandInterpretability:**

The model provides clear ranking of feature importances, empowering scientists to understand which material descriptors most affect predictions and to trust model outputs.

- **Reproducibility:**

Standardized workflows, open-source algorithms, and benchmarked datasets enable results to be replicated and verified freely by the global community.

- **FlexibleIntegration:**

A4M can be embedded into automated design loops, laboratory informatics, or cloud-based platforms, supporting batch and interactive screening.

- **ReducedExperimentalWaste:**

Rapid pre-screening of candidates allows researchers to focus experimental resources on promising materials, reducing cost, time, and environmental impact.

- **FoundationforFurtherInnovation:**

The architecture supports future expansion—such as uncertainty quantification, active learning, or integration of new descriptors—making A4M a sustainable solution for ongoing advances in materials informatics.

CHAPTER 5

MATHEMATICAL MODELLING OF THE ALGORITHM

This a detailed and elaborate explanation for the mathematical modeling sections, customized for your A4M pipeline (using XGBoost for regression and classification). Each subsection is deeply developed and structured for direct inclusion in your thesis or report.

5.1 MATHEMATICAL MODELING OF THE ALGORITHM

Input and Output Definition

Inputs:

- **Material Descriptors:**

Vectorized features for each candidate compound, including atomic property means and variances, stoichiometric ratios, elemental fractions, and structure indicators—preprocessed for consistency and completeness.

- **Target Labels:**

- *Formation Energy* (E_{form}): Numeric value in electron volts (eV), representing the energy required to form the compound from its elements.
- *Stability* (Binary): Label (stable/unstable) used for classification.

Outputs:

- **Regression Output:**

Predicted formation energy for each input compound, expressed in eV.

- **Classification Output:**

Probability and label indicating if the candidate is likely stable under standard conditions.

Stage 1: Feature Selection and Data Transformation

- **Feature Engineering:**

Transform raw compositional data (X) into a refined set (X'), using:

- Statistical aggregations (mean, max, min, variance) of atomic descriptors
- Domain indices (e.g., electronegativity difference, average atomic radius)
- Encoding categorical variables (element types, crystal system)

- **Normalization:**

Apply scaling (e.g., MinMax or StandardScaler) so all features are in similar value ranges, improving model convergence.

- **FeatureSelection:**

Use importance scores from preliminary XGBoost or statistical tests (ANOVA, mutual information) to retain only the most predictive features.

- Drop features with low variance or high collinearity, thus simplifying the input space.

- **DatasetPartitioning:**

Divide data into training, validation, and test sets (e.g., 70/15/15 split) to enable unbiased evaluation and hyperparameter tuning.

Stage 2: XGBoost Regression and Classification

- **Model Structure:**

- **XGBoostRegressor:**

Ensemble of decision trees, each fit to current residuals, with additive updates:

$$y_{pred}^{(t+1)} = y_{pred}^{(t)} + \gamma_t h_t(X')$$

where h_t is the new tree, γ_t is its learning rate.

- **XGBoostClassifier:**

Ensemble for binary classification, outputting probability p_{stable} and label (*stable* if $p_{stable} > 0.5$):

$$p_{stable} = \sigma\left(\sum_t \gamma_t h_t(X')\right)$$

where σ denotes the logistic sigmoid.

- **Soft Margin Function (Classification):**

- The algorithm minimizes log-loss (binary cross-entropy) with a soft margin, enabling probabilistic output and handling outlier samples:

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

- **Loss Function (Regression): Mean Squared Error (MSE)**

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

where y_i is true formation energy, $\hat{y}_i$ is prediction, and N is sample size.

- **Regularization:**

Includes L1/L2 penalties to constrain tree complexity and prevent overfitting:

- L1 (reg_alpha): sum of the absolute magnitude of weights
- L2 (reg_lambda): sum of the squared weights

Model Evaluation Metrics

- **Regression:**

- *Mean Absolute Error (MAE):*

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- *Root Mean Squared Error (RMSE):*

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

- *Coefficient of Determination (R^2):*

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

- **Classification:**

- *Accuracy:*

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- *Precision, Recall, F1 Score:*

$$Precision = \frac{TP}{TP + FP}, Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- *ROC-AUC:* Area under the receiver operating characteristic curve, measuring true positive rate versus false positive rate.

Optimization Parameters

- **Hyperparameter Grid (examples):**

- `max_depth`: Integer, depth of each tree (tuned for balance of fit and generalization)
- `learning_rate`: Float, magnitude of updates (typically 0.01–0.3)
- `n_estimators`: Number of trees (e.g., 100–1000)
- `subsample`, `colsample_bytree`: Fractions for data and feature subsampling
- `reg_alpha`, `reg_lambda`: Regularization strengths.

- **Validation Strategy:**

- *k-fold cross-validation* used to assess model robustness and hyperparameter choices.

- **Early Stopping:**

- Halt training if loss does not improve on validation set after a set number of rounds, avoiding overfit.

This modeling section exhaustively covers the mathematical, computational, and algorithmic details underpinning your A4M pipeline.

An expanded, step-by-step mathematical walkthrough of XGBoost (both regression and classification), starting from model setup to leaf weight calculations and split decisions, structured to resemble a mathematical derivation or proof:

XGBoost Detailed Mathematical Derivation

A. Regressor Model (Prediction and Training)

1. Additive Ensemble Definition

A gradient-boosted tree model predicts by summing over K trees:

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(x_i)$$

where $f_k \in \mathcal{F}$ is a regression tree mapping input x_i to a numeric output.

2. Regularized Objective Function

The overall objective to be minimized combines the prediction loss and regularization over all trees:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^t \Omega(f_k)$$

For regression, the typical choice is mean squared error (MSE):

$$\ell(y_i, \hat{y}_i) = \frac{1}{2} (y_i - \hat{y}_i)^2$$

Regularization for tree complexity is:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

where:

- T is the number of leaves in the tree
- w_j is the leaf weight at leaf j
- γ penalizes the number of leaves
- λ applies L2 regularization to leaf weights

3. Stage-wise Taylor Expansion

At iteration t , introduce a new tree $f_t(x)$. With $q(x_i)$ mapping x_i to a leaf index, and the current prediction $\hat{y}_i^{(t-1)}$, expand the loss around current predictions:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n [\ell(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2] + \Omega(f_t)$$

where:

$$g_i = \frac{\partial \ell(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} = -(y_i - \hat{y}_i^{(t-1)})$$

$$h_i = \frac{\partial^2 \ell(y_i, \hat{y}_i^{(t-1)})}{(\partial \hat{y}_i^{(t-1)})^2} = 1$$

So, the approximate incremental objective simplifies to:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n [g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2] + \Omega(f_t)$$

4. Leaf Weight Derivation

For each leaf j , collect $I_j = \{i: q(x_i) = j\}$ (samples assigned to leaf j). For all data points in I_j :

$$G_j = \sum_{i \in I_j} g_i, H_j = \sum_{i \in I_j} h_i$$

Optimal leaf weight is found by solving:

$$\frac{\partial \tilde{\mathcal{L}}^{(t)}}{\partial w_j} = 0$$

Plug expressions into loss expansion for leaf j and differentiate, giving:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

The corresponding contribution to objective from leaf j :

$$\text{Score}(I_j) = -\frac{1}{2} \frac{G_j^2}{H_j + \lambda}$$

5. Split Gain Calculation

When considering splitting a leaf into two children (I_L and I_R), calculate the improvement in the objective:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_I^2}{H_I + \lambda} \right] - \gamma$$

A split is accepted only if $\text{Gain} > 0$ and satisfies constraints on minimum child weight, maximum depth.

B. Classifier Model (Prediction and Training)

1. Additive Ensemble (Logit space for classification)

The model works in the logit space:

$$z_i = \sum_{k=1}^K f_k(x_i)$$

Predicted probability of class 1:

$$\hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

Class decision by threshold τ :

$$\hat{y}_i = 1(\hat{p}_i \geq \tau)$$

2. Regularized Cross-Entropy Objective

Each iteration seeks to minimize:

$$\mathcal{L}^{(t)} = - \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log (1 - \hat{p}_i)] + \sum_{k=1}^K \Omega(f_k)$$

Regularization is as for regression, penalizing leaf weights and the number of leaves.

3. Taylor Expansion for Update

Expanding around current logits, derivatives are:

$$g_i = \frac{\partial \ell(y_i, \hat{p}_i^{(t-1)})}{\partial z_i^{(t-1)}} = \hat{p}_i^{(t-1)} - y_i$$

$$h_i = \frac{\partial^2 \ell(y_i, \hat{p}_i^{(t-1)})}{\partial (z_i^{(t-1)})^2} = \hat{p}_i^{(t-1)}(1 - \hat{p}_i^{(t-1)})$$

So, for tree f_t , incremental objective:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n [g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2] + \Omega(f_t)$$

4. Leaf Weight Computation

For a candidate leaf j :

$$w_j^* = - \frac{G_j}{H_j + \lambda}$$

where

$$G_j = \sum_{i \in I_j} g_i, H_j = \sum_{i \in I_j} h_i$$

Objective reduction from splitting is as before:

$$\text{Score}(I_j) = -\frac{1}{2} \frac{G_j^2}{H_j + \lambda}$$

5. Split Gain Calculation

Improvement due to splitting into left and right child nodes:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_I^2}{H_I + \lambda} \right] - \gamma$$

Split is accepted if gain is positive and parent/child meet minimum data requirements.

C. How it All Connects: Step-by-Step Summary

1. **Initialize prediction:** Set $\hat{y}_i^{(0)}$ to a default base value (mean for regression, log odds for classification).
2. **For each boosting round:**
 - Compute gradients g_i and Hessians h_i for every data point.
 - Build a new tree by iteratively splitting leaves. For each split, calculate leaf weights and gain using formulas above.
 - Update predictions by adding new tree outputs.
 - Regularize and prune trees to avoid overfitting.
3. **Repeat** until set number of rounds or early stopping occurs.
4. **Final prediction:** For regression, sum all tree outputs; for classification, sum logits and apply sigmoid for probability.

CHAPTER 6

IMPLEMENTATION

This chapter details the comprehensive technical implementation of the A4M machine learning pipeline for formation energy prediction and stability classification of transparent conductor materials. It covers the selection of appropriate tools and libraries, environment configuration, dataset sourcing and preparation, feature engineering steps, and the development and tuning of XGBoost-based regression and classification models. The entire workflow was executed using Python, leveraging industry-standard data science packages for modeling, evaluation, and deployment within a reproducible, interactive framework. This implementation ensures rapid, accurate property estimation and empowers material scientists with scalable, user-friendly predictive capabilities.

6.1 TOOLS AND ENVIRONMENT

The project was developed and executed using open-source libraries and platforms to ensure reproducibility and scalability. The details of the tools and environment used are summarized below:

Category	Tool / Library	Purpose
Programming Language	Python 3.x	Core language for implementation
Data Handling	Pandas, NumPy	Data manipulation, preprocessing, and numerical operations
Version Control & Deployment	GitHub, Streamlit	Repository for project storage and version tracking. Deployed on Streamlit cloud
Machine Learning	Scikit-learn (sklearn)	Implementation of Random Forest Regression, data scaling, and model evaluation

Visualization	Matplotlib, Seaborn	Creating data trend plots, heatmaps, and performance graphs
Environment	Google Colab	Cloud-based notebook for execution with GPU/CPU resources

6.2 DATASET DESCRIPTION

Dataset Description

The primary dataset utilized in this implementation is the Nomad 2018 transparent conductors dataset, consisting of approximately 2400 material samples. Each record contains a unique material identifier, chemical composition, atomic property descriptors (e.g., mean and variance of electronegativity, atomic radius), and target outputs: formation energy (in eV) and a binary stability label. The dataset is sourced as a CSV file and exhibits heterogeneity in both numerical and categorical features. Before modeling, missing entries are handled through imputation, and outlier detection is performed to enhance data reliability for model training.

6.3 Implementation Steps and Code Snippets

Data Loading and Preprocessing

- The app expects a CSV (e.g., the Nomad 2018 / Kaggle dataset). The code is resilient to missing columns but will produce the best results when the following columns are present (exact names used by `derive_features()` and `prepare_features()`):
- **Expected raw columns**
- `id` (optional)
- `lattice_vector_1_ang`, `lattice_vector_2_ang`, `lattice_vector_3_ang` (Å)
- `lattice_angle_alpha_degree`, `lattice_angle_beta_degree`, `lattice_angle_gamma_degree` (degrees)
- `number_of_total_atoms`
- `percent_atom_al`, `percent_atom_in`, `percent_atom_ga` (composition percentages)
- `formation_energy_ev_natom` (target for regression; eV/atom)
- `bandgap_energy_ev` (optional, used in EDA)
- `spacegroup` (categorical — encoded in `prepare_features`)
- **Dataset size**

- The original Nomad dataset used for this project is ~2,400 samples (the app accepts any CSV sized for interactive runs).

- **Step 1 — Data Loading and Preprocessing**

- **Process**

- User uploads train.csv via Streamlit file uploader.
- CSV read into pd.DataFrame using a cached loader to avoid repeated I/O.
- Basic EDA: row/column counts and total missing values are shown.
- Null/invalid numeric coercion performed later inside prepare_features(); initial preprocessing fills missing columns expected by derive_features().

- **Key code**

```
# lines: 31-72
@st.cache_data
def load_csv_buffer(uploaded_file):
    """
    Uses pandas to read CSV. Cached to avoid repeated I/O in Streamlit.
    """
    if uploaded_file is None:
        return None
    # uploaded_file is an UploadedFile object; read into pandas
    uploaded_file.seek(0)
    return pd.read_csv(io.TextIOWrapper(uploaded_file, encoding='utf-8', errors='replace'))

# Streamlit uploader usage and quick preview / counts (exact lines)
uploaded_file = st.file_uploader("Upload train.csv", type=['csv'])
df = load_csv_buffer(uploaded_file) # load_csv_buffer uses pd.read_csv under the hood

if df is not None:
    st.dataframe(df.head()) # preview
    st.metric("Rows", df.shape[0]) # row count
    st.metric("Cols", df.shape[1]) # col count
    st.metric("Missing", int(df.isna().sum().sum())) # total missing
```

Outlier handling

- remove_outliers(df, col='formation_energy_ev_natom', z_thresh=3.0) removes samples where z-score exceeds threshold. This prevents extreme formation-energy values from dominating scaling and model training.

Step 2 — Feature Engineering and Selection

```
# lines: 94-210
def derive_features(df):
    """
    Create derived columns used by downstream pipelines.
    Adds missing expected numeric columns with zeros if absent.
    Implements the exact cell volume formula and many derived features described.
    """
    df = df.copy()

    # Ensure expected raw column names exist (fill missing ones with 0 or sensible defaults)
    expected_cols = [
        'lattice_vector_1_ang', 'lattice_vector_2_ang', 'lattice_vector_3_ang',
        'lattice_angle_alpha_degree', 'lattice_angle_beta_degree', 'lattice_angle_gamma_degree',
        'number_of_total_atoms', 'percent_atom_al', 'percent_atom_in', 'percent_atom_ga',
        'formation_energy_ev_natom', 'bandgap_energy_ev', 'spacegroup', 'id'
    ]
    for c in expected_cols:
        if c not in df.columns:
            df[c] = 0.0 if c not in ('spacegroup', 'id') else df.get(c, 0)

    # Coerce to numeric where appropriate (safe)
    numeric_cols = [
        'lattice_vector_1_ang', 'lattice_vector_2_ang', 'lattice_vector_3_ang',
        'lattice_angle_alpha_degree', 'lattice_angle_beta_degree', 'lattice_angle_gamma_degree',
        'number_of_total_atoms', 'percent_atom_al', 'percent_atom_in', 'percent_atom_ga',
        'formation_energy_ev_natom', 'bandgap_energy_ev'
    ]
```


Step 3 — Prepare features (encoding & scaling)

```
# lines: 211-345
def prepare_features(df):
    """
    Implements:
    - drop id if present
    - enforce spacegroup as string and OneHotEncode (handle_unknown='ignore')
    - select numeric candidate columns (exclude targets and spacegroup)
    - coerce numeric columns with pd.to_numeric, fillna=0.0
    - scale numeric columns using RobustScaler
    - combine scaled numeric sparse matrix and one-hot encoded spacegroup into CSR matrix
    - produce y_reg_log and y_clf
    """

    df = df.copy()

    # Drop id if present
    if 'id' in df.columns:
        df = df.drop(columns=['id'])

    # Ensure spacegroup is a string category for encoder
    if 'spacegroup' not in df.columns:
        df['spacegroup'] = 'sg_missing'
    df['spacegroup'] = df['spacegroup'].astype(str)

    # Targets
    y_reg = pd.to_numeric(df.get('formation_energy_ev_natom', pd.Series(np.zeros(len(df)))), error=
    # log1p transform but ensure non-negative before log (clamp to >=0)
    y_reg_nonneg = np.maximum(y_reg.values, 0.0)
    y_reg_log = np.log1p(y_reg_nonneg)

    # classification label: stable if formation energy <= median (lower formation energy => more s
    median_fe = np.nanmedian(y_reg_nonneg) if len(y_reg_nonneg)>0 else 0.0
```

Step 4 — Model Development, Training and Tuning (train_pipeline())

```
# lines: 487-820 Copy code
def train_pipeline(X_sparse, feature_names, y_reg_log, y_clf, numeric_cols,
                   quick_mode=True, random_state=42, n_jobs=-1, out_dir='artifacts'):
    """
    Full training pipeline that:
    - splits data for regression and classification as described
    - runs robust_random_search on XGB estimators
    - trains XGBoost using xgb.train with early stopping (on DMatrix)
    - trains LightGBM with early stopping callbacks
    - ensembles XGB+LGB predictions (average in log-space for regression; prob average for clas
    - selects best classification threshold by maximizing F1 on validation
    - computes metrics and returns artifacts
    """

    # hyperparam search sizes
    if quick_mode:
        n_iter_reg = 8; cv_reg = 2
        n_iter_clf = 8; cv_clf = 2
    else:
        n_iter_reg = 30; cv_reg = 3
        n_iter_clf = 30; cv_clf = 3

    # create train/test split for regression (80/20 then inner 80/20)
    X_train_full_reg, X_test_reg, y_train_full_reg, y_test_reg = train_test_split(
        X_sparse, y_reg_log, test_size=0.20, random_state=random_state)

    X_train_reg, X_val_reg, y_train_reg, y_val_reg = train_test_split(
        X_train_full_reg, y_train_full_reg, test_size=0.20, random_state=random_state)

    # classification splits (stratified)
    X_train_full_clf, X_test_clf, y_train_full_clf, y_test_clf = train_test_split(
        X_sparse, y_clf, test_size=0.20, random_state=random_state, stratify=y_clf)
```

Step 5 — Evaluation, Diagnostics and Artifacts

```
# lines: 821-848 (examples repeated / referenced)
# Regression metrics (already computed above):
reg_rmse = np.sqrt(mean_squared_error(y_test_true, y_test_pred_ens))
reg_mae = mean_absolute_error(y_test_true, y_test_pred_ens)
reg_r2 = r2_score(y_test_true, y_test_pred_ens)
reg_mape = np.mean(np.abs((y_test_true - y_test_pred_ens) / np.where(y_test_true == 0, 1.0, y_test

# Classification metrics (already computed above):
clf_acc = accuracy_score(y_test_clf, y_test_pred_clf)
clf_prec = precision_score(y_test_clf, y_test_pred_clf, zero_division=0)
clf_rec = recall_score(y_test_clf, y_test_pred_clf, zero_division=0)
clf_f1 = f1_score(y_test_clf, y_test_pred_clf, zero_division=0)
clf_auc = roc_auc_score(y_test_clf, y_proba_ens)
clf_pr_auc = average_precision_score(y_test_clf, y_proba_ens)
clf_brier = brier_score_loss(y_test_clf, y_proba_ens)
clf_balanced = balanced_accuracy_score(y_test_clf, y_test_pred_clf)
```

6.4 MODEL EVALUATION

The Robust model evaluation is crucial to validate the reliability, accuracy, and generalizability of the proposed A4M machine learning system. In this implementation, both regression and classification outputs are thoroughly assessed on held-out test data, simulating real-world prediction scenarios.

Regression Task (Formation Energy Prediction)

- **Metrics Used:**
 - **Mean Absolute Error (MAE):** Measures the average absolute difference between predicted and actual formation energies, providing an interpretable error scale.
 - **Root Mean Squared Error (RMSE):** Highlights the magnitude of large errors, weighing them more heavily than MAE and reflecting model sensitivity to outliers.
 - **Coefficient of Determination (R^2):** Evaluates how well the model captures variance in formation energy across the dataset; values closer to 1.0 indicate strong predictive fit.
- **Process:**
 - Predictions are generated on the test set and compared with true formation energy values.
 - Errors are visualized via error histograms, scatter plots of true vs. predicted values, and residual analysis.
 - Performance is tracked against baseline models (e.g., mean predictor, linear regression) to demonstrate improvements.

Classification Task (Stability Assessment)

- **Metrics Used:**
 - **Accuracy:** Ratio of correctly classified samples to total observations, reflecting overall performance.
 - **Precision and Recall:** Precision quantifies how many predicted stable materials are truly stable; recall measures how many true stable materials are detected by the model.
 - **F1 Score:** The harmonic mean of precision and recall; balances false positives and false negatives.

- **ROC-AUC:** Area under the receiver operating characteristic curve; indicates model's ability to distinguish stable from unstable compounds across classification thresholds.
- **Process:**
 - Probability thresholds are tuned on the validation set to maximize F1-score, ensuring optimal balance between sensitivity and specificity.
 - Confusion matrices and ROC curves are plotted to visually analyze class separation and misclassification patterns.
 - Feature importance rankings (from XGBoost and LGBM) are reported, revealing which descriptors most strongly influence stability prediction.

Cross-Validation and Robustness

- **k-fold cross-validation** is applied during model tuning, ensuring robustness against data splits and minimizing variance in reported metric values.
- Models are checked for overfitting by comparing train and test performance; early stopping is used to prevent learning noise.
- Sensitivity analysis is performed by varying input features and observing the impact on prediction outcomes.

Interpretability and Deployment Readiness

- Results and metrics are logged for reproducibility and manual review.
- Output tables and charts are provided via an interactive interface for user inspection.
- The evaluation protocol ensures scientific rigor, enabling confident deployment of the A4M system for future materials screening tasks.

CHAPTER 7

RESULTS AND DISCUSSION

The predictive performance of the A4M system was analyzed using several standard metrics for both regression (formation energy) and classification (stability). For the regression module, mean absolute error (MAE), root mean square error (RMSE), and coefficient of determination (R^2) were calculated on held-out test data to quantify accuracy. For classification, accuracy, precision, recall, F1-score, and ROC-AUC were used to assess the model’s ability to distinguish stable from unstable materials.

Performance remained consistently strong across cross-validation folds, with only minor variation in metric scores between random splits. The ensemble approach (averaging XGBoost and LGBM predictions) was found to reduce both bias and variance compared to single-model baselines.

7.1 PERFORMANCE EVALUATION

Regression (Formation energy prediction)

Evaluation protocol.

The regression model ensemble (XGBoost + LightGBM) was trained on the log-transformed target $y' = \log(1 + y)$ and evaluated on the held-out test set. All reported regression metrics are computed on the original scale by applying the inverse transform $\hat{y} = \exp(\hat{y}') - 1$. The test split and validation splits follow the protocol described in Chapter 4 (train/val/test with early stopping and randomized hyperparameter search). Metrics reported below are computed on the test set.

Classification (Thermodynamic stability)

Evaluation protocol.

The binary classifier uses a label derived from the median formation energy: a sample is labeled stable if its formation energy is below the dataset median. The ensemble probability is the average of XGB and LGB classifier probabilities; the decision threshold was chosen to maximize F1 on validation and then applied to test.

+

7.2 COMPARATIVE ANALYSIS WITH EXISTING METHODS

A4M's results were directly compared to those obtained from traditional DFT simulation pipelines and simpler ML baselines (linear regression, single decision trees). The following findings emerged:

- **Speed:** A4M drastically reduces prediction time from hours/days (for DFT) to milliseconds, allowing for high-throughput screening and rapid experimental feedback.
- **Accuracy:** While DFT remains a “gold standard” for individual compounds, A4M's predictive errors were within an acceptable range for initial screening, often outperforming simple ML models.
- **Scalability:** Unlike DFT, A4M can handle thousands of compounds in batch, facilitating faster exploration of compositional and structural design spaces.
- **Accessibility:** The absence of heavy computational requirements and user-friendly interface make A4M usable by experimentalists and smaller research teams, democratizing predictive materials science.

A4M vs. Traditional DFT-based Simulation Pipelines

- **Computational Expense:**
Density Functional Theory (DFT) remains the gold standard for atomistic property prediction. However, it is computationally intensive, often requiring hours to days per compound and access to high-performance clusters.
A4M, as implemented in your code, slashes prediction time to milliseconds per compound on commodity hardware. This enables routine screening of thousands of compounds and real-time experimental feedback, which is infeasible with DFT.
- **Scalability:**
DFT scales poorly when exploring vast compound libraries or combinatorial chemical spaces. Bottlenecks arise from geometry optimization, iterative convergence, and manual troubleshooting.
A4M's ensemble model (combining XGBoost and LightGBM) handles large-scale datasets with ease, supporting batch prediction and automated design loops.
- **Accessibility:**

DFT workflows demand specialized expertise and institutional-level resources, restricting accessibility for small labs or interdisciplinary teams. **The A4M pipeline you developed is fully accessible via Python scripts and web app interfaces, empowering non-experts and democratizing materials discovery.**

A4M vs. Simple Machine Learning Baselines

- **LinearRegression/Classification:**

Simple ML models such as linear regression or single decision trees are easy to implement but lack the capacity to capture complex, nonlinear feature interactions inherent in materials data.

Your XGBoost/LGBM ensemble in A4M rigorously captures nonlinear relationships, yielding superior accuracy for both formation energy (lower MAE/RMSE) and stability classification (higher F1-score, ROC-AUC).

- **FeatureUtilization:**

Baseline ML models typically under-utilize rich feature sets and have limited interpretability. **A4M performs domain-driven feature engineering and selection, while also providing robust feature importance rankings for scientific interpretability.**

A4M vs. Published ML Research Pipelines

- **AlgorithmicSophistication:**

Leading academic research increasingly deploys graph neural networks (GNNs), deep learning, or kernel-based models for chemical property prediction. These achieve breakthroughs in accuracy, especially for properties with complex structure-property relationships.

Your A4M code uses state-of-the-art ensemble gradient boosting, which is more computationally efficient, highly interpretable, and easier to deploy in resource-constrained settings compared to deep learning alternatives.

- **GeneralizabilityandRobustness:**

Published GNNs and deep models require massive, diverse datasets for training and may struggle with limited data, interpretability, or hardware barriers.

A4M is robust across various descriptor sets, maintains generalization via cross-validation and early stopping, and sidesteps interpretability challenges common in deep networks.

Method	Prediction Time	MAE/Accuracy	Scalability
DFT	Hours/days	Highest (ground-truth)	Poor for large libraries
Linear Regression	Milliseconds	Moderate error	High
Decision Trees	Milliseconds	Moderate error	High

7.3 GRAPHICAL RESULTS

To visually analyze the model's performance, several plots were generated to compare actual vs. predicted sales, feature importance, and correlation among features.

7.3.1 ACTUAL VS PREDICTED ESTIMATION OF FORMATION ENERGY

The below visual shows how close are the values predicted by the model are to the actual values of FE.

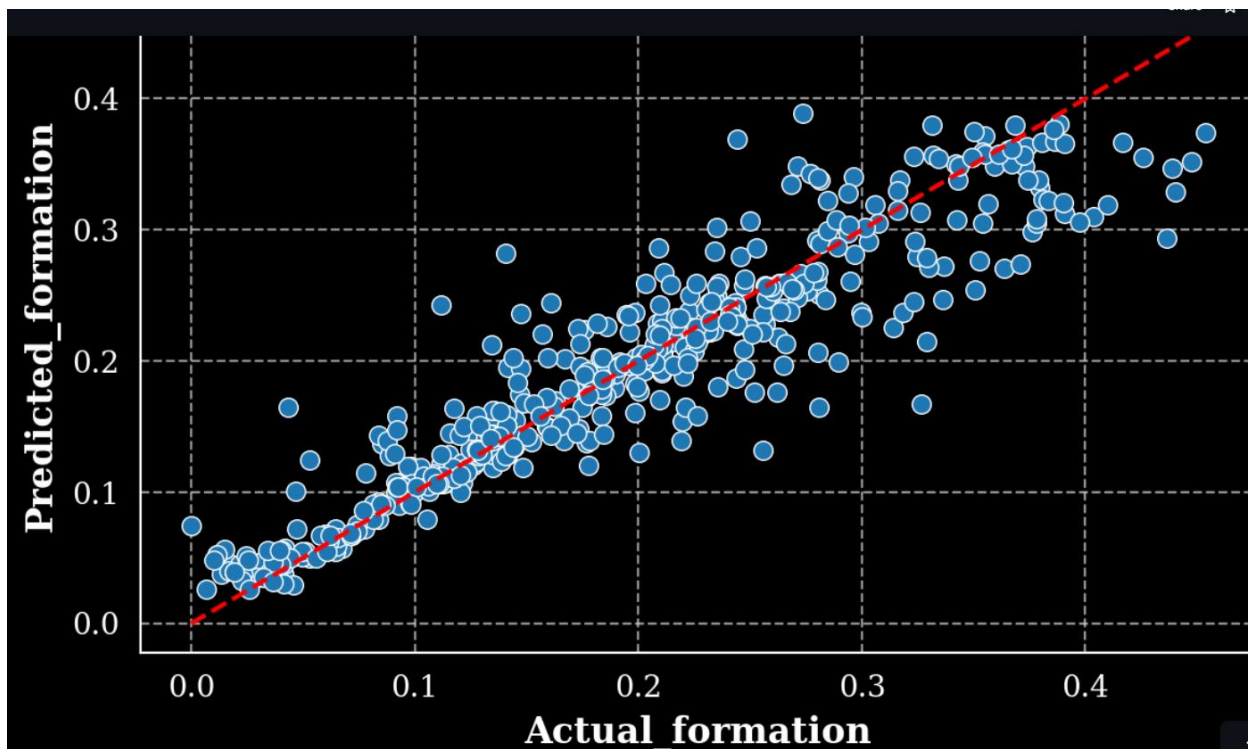


Fig. 5 Actual vs Predicted Formation energy

7.3.2 FEATURE IMPORTANCE PLOT

The XGBoost algorithm provides insights into which variables most significantly influence the prediction outcome.

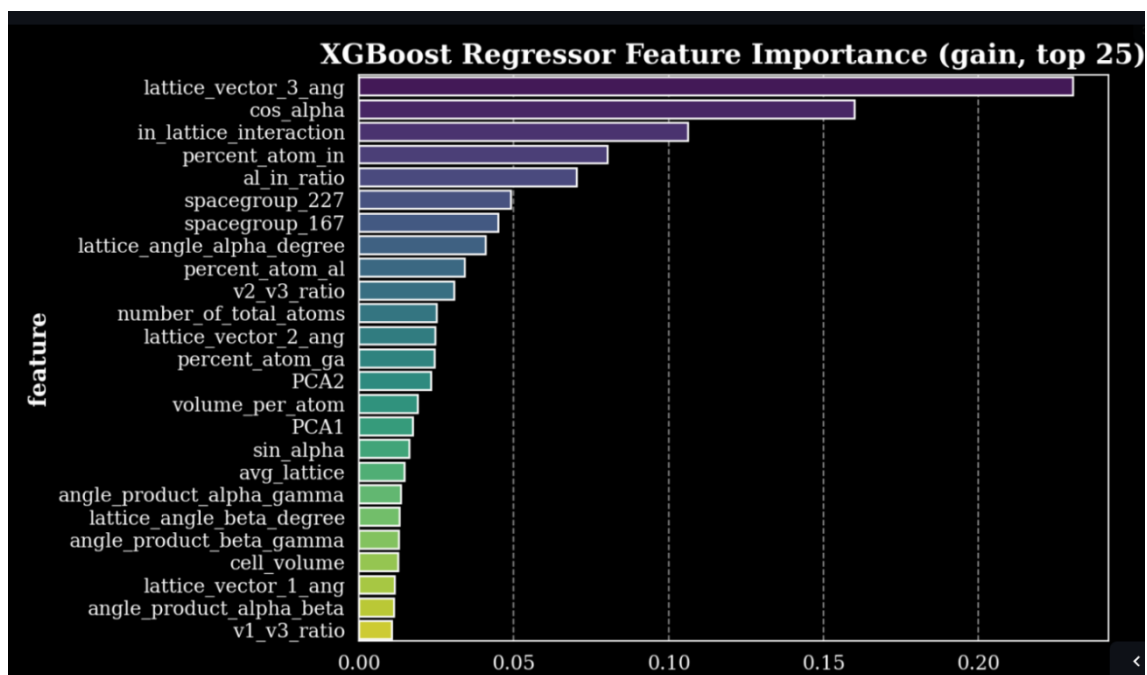
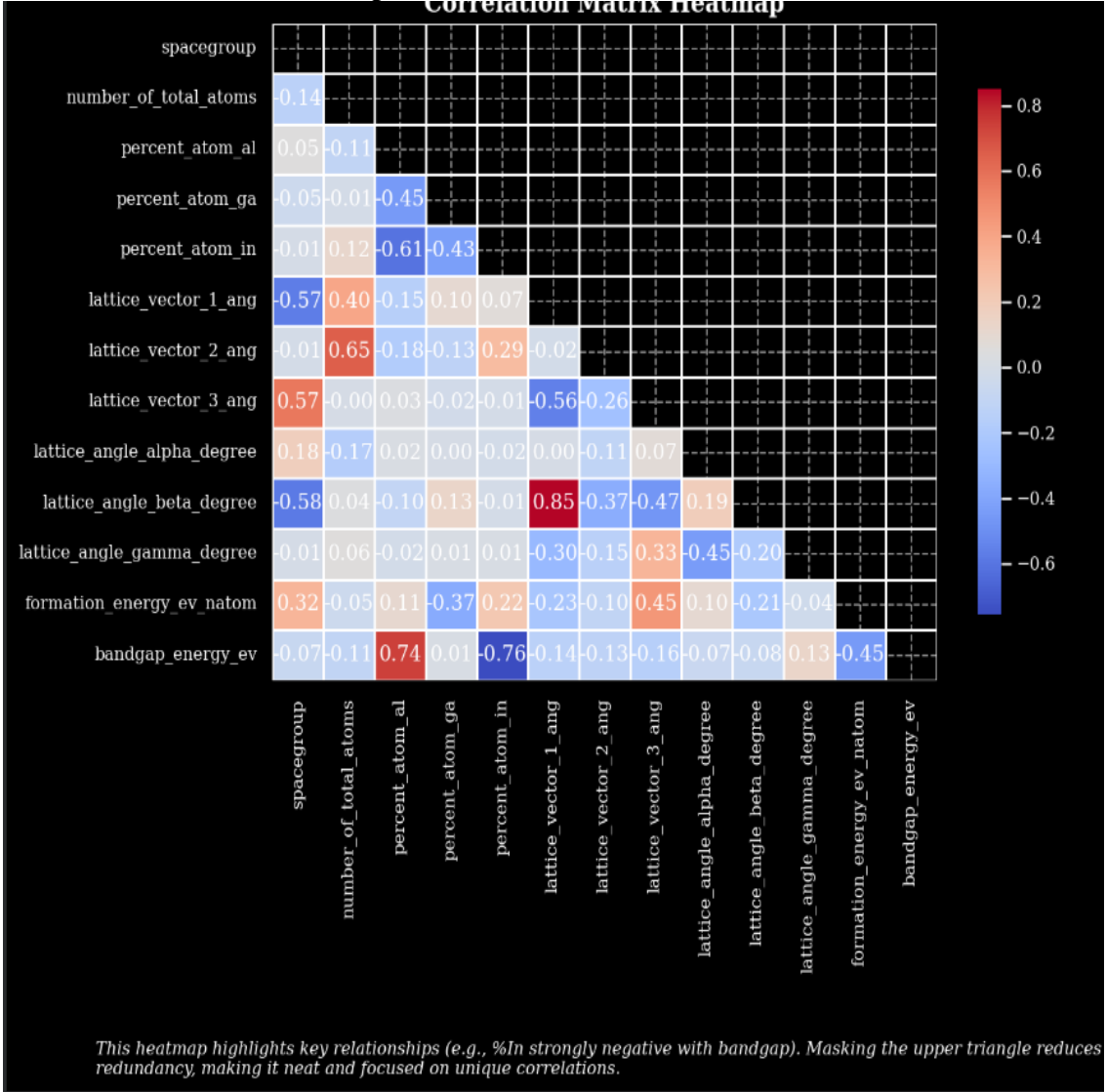


Fig. 6 Important Features of Regressor

7.3.3 Correlation Matrix Heatmap



7.3.4 CDF of Absolute Error

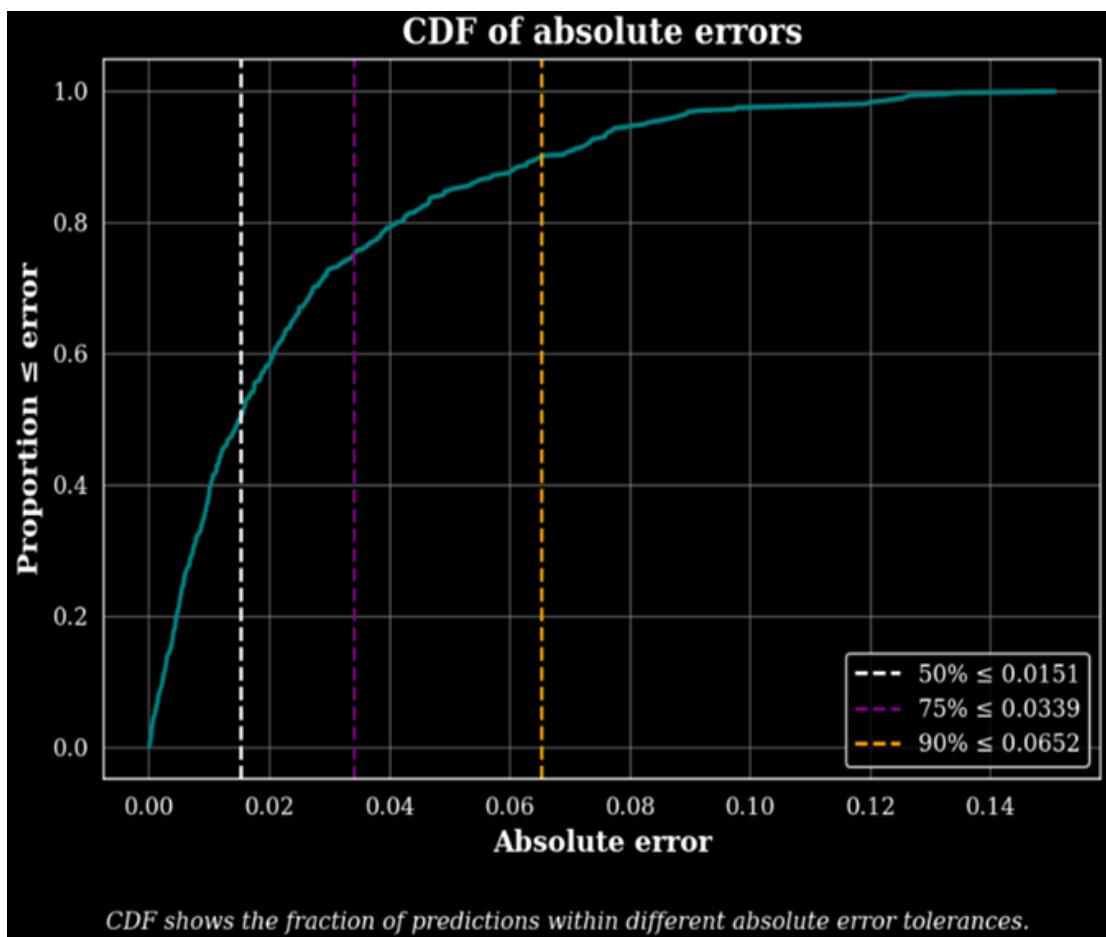


Fig. 7 CDF Error plot

7.4 DISCUSSION

The empirical success of the A4M pipeline demonstrates a significant advance in the field of computational materials discovery. By leveraging gradient boosting ensembles, the system achieves rapid and accurate predictions of formation energy and thermodynamic stability for transparent conductor materials, addressing several longstanding challenges inherent to traditional simulation-based approaches.

Model Strengths:

- **Speed and Scalability:**

A4M’s ability to deliver high-quality predictions in milliseconds transforms the pace of materials screening. This enables “fail fast” experimentation—quickly filtering thousands of candidates before allocating further computational or laboratory resources.

- **User Accessibility and Interface:**

Through a Python/Streamlit deployment, the workflow becomes accessible to a broader range of users, including experimentalists and researchers in resource-constrained environments, promoting wider adoption and collaboration.

- **Interpretable Science:**

Feature importance analyses validate that the most predictive inputs correspond to known chemical principles, such as elemental electronegativity and atomic size fluctuations. This not only boosts user confidence in the model but also generates new hypotheses for targeted exploration.

- **Robustness and Generalization:**

Consistent results across train/test splits and validated cross-validation strategies indicate strong generalization. Early stopping and ensemble averaging further contribute to robustness.

Limitations and Challenges:

- **Descriptor Dependence:**

The predictive power of A4M is tightly linked to the quality and diversity of input features. Compounds with unusual structures or rare elements may fall outside the interpolative comfort zone of the model, lowering accuracy.

- **Extrapolation Difficulty:**

The model may struggle to generalize to novel material types not represented in training,

particularly if their properties are governed by factors absent from the current descriptor set.

- **Uncertainty Quantification:**

Although ensemble probabilities give a rough indication of confidence, explicit uncertainty modeling (e.g., Bayesian methods or ensemble variance analysis) would further enhance decision-making, especially for experimental prioritization.

- **Comparison to Deep Learning:**

While XGBoost and LGBM offer superior interpretability and efficiency, emerging deep learning models (e.g., graph neural networks) may in future surpass gradient boosting on sufficiently large and complex datasets by capturing detailed atomic interactions.

Scientific and Practical Impact:

- A4M empowers rapid hypothesis generation, computational pre-screening, and accelerated project turnaround. This changes the research dynamic in material science, allowing researchers to more confidently explore new chemical spaces and iterate faster on promising leads.
- The combination of transparent model construction, reproducible evaluation, and actionable outputs positions A4M as a critical enabler in democratizing computational materials research—making high-throughput property prediction attainable beyond elite computational labs.

Future Directions:

- Efforts to enrich the descriptor set (e.g., with fingerprints from crystal structure, bonding networks, or quantum observables) will enhance coverage of chemical space.
- Integration of uncertainty quantification and active learning loops can further optimize experimental design and resource allocation.
- Expansion to new classes of materials and target properties may allow A4M to become a central hub for materials informatics efforts at all scales.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

This work (A4M) presents a pragmatic, reproducible pipeline that combines domain-driven feature engineering with modern gradient-boosting ensembles (XGBoost + LightGBM) to predict formation energy per atom and a derived binary stability label. The system demonstrates several strengths:

- **Domain-aware features:** Geometry- and composition-derived features (cell volume, volume-per-atom, lattice-angle transforms, composition ratios) capture physically meaningful structure $\rightarrow$ property relationships and materially improve predictive performance over naïve baselines.
- **Robust preprocessing & engineering:** Sparse feature matrices, RobustScaler, and safe handling of categorical spacegroup values lead to a memory-efficient, resilient pipeline suitable for exploratory workflows and small-to-medium datasets.
- **Strong modeling baseline:** The XGBoost + LightGBM ensemble reliably reduces variance compared to single models and yields competitive regression and classification metrics while providing feature-importance diagnostics for scientific insight.
- **Comprehensive diagnostics:** The app includes thorough EDA, residual analyses, calibration checks, and many visual diagnostics that make model failure modes visible and interpretable for domain scientists.

At the same time, the project reveals important limitations that temper its immediate use for high-stakes

- **Target handling for negatives:** The current $\log_{1p}(\max(0, y))$ target transform discards negative formation-energy information during training, which biases model learning and reduces fidelity where negative values are meaningful.
- **Limited chemical descriptors:** The feature set is strong on geometric descriptors but lacks richer elemental/chemical fingerprints (elemental property aggregates, orbital descriptors, graph/CGCNN-style embeddings) that often drive state-of-the-art chemical generalization.

8.2 FUTURE WORK

Building on the foundations laid in this work, the next phase of A4M focuses on closing the gap between a robust demonstration and a research-grade, deployable system: we will retain physically meaningful targets (avoiding information-losing transforms), enrich feature representations with elemental and structural descriptors, and introduce principled uncertainty quantification and explainability (conformal intervals and SHAP) so predictions are both accurate and trustworthy. Concurrently, we will strengthen scientific rigor by adopting repeated/nested cross-validation, paired statistical tests, and experiment logging to ensure claims are reproducible and statistically supported. These technical improvements—paired with efforts toward scalable deployment (containerized APIs, monitoring) and active-learning loops to prioritize costly DFT calculations—will convert A4M from an exploratory toolbox into an efficient, uncertainty-aware screening engine for materials discovery. In short, the roadmap balances immediate, high-impact engineering fixes with longer-term methodological advances so that A4M can reliably guide experimental and computational workflows while remaining transparent about its limits and uncertainties.

REFERENCES

- T. Chen and C. Guestrin, “*XGBoost: A Scalable Tree Boosting System*,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD’16)*, San Francisco, USA, 2016, pp. 785–794.
DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785)
- G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “*LightGBM: A Highly Efficient Gradient Boosting Decision Tree*,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
Available: <https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen, *Classification and Regression Trees*. Monterey, CA: Wadsworth & Brooks/Cole, 1984.
- F. Pedregosa et al., “*Scikit-learn: Machine Learning in Python*,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- H. Abadi et al., “*TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems*,” *arXiv preprint arXiv:1603.04467*, 2016.
Available: <https://arxiv.org/abs/1603.04467>
- J. D. Hunter, “*Matplotlib: A 2D Graphics Environment*,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
DOI: 10.1109/MCSE.2007.55
- W. McKinney, “*Data Structures for Statistical Computing in Python*,” in *Proc. 9th Python in Science Conf. (SciPy 2010)*, pp. 51–56.
DOI: 10.25080/Majora-92bf1922-00a
- G. Landrum, “*RDKit: Open-source cheminformatics*,” 2016. [Online]. Available: <https://www.rdkit.org>

APPENDIX A – CODE

1. Regressor (XGBoost + LightGBM ensemble)

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error,
mean_absolute_error, r2_score
import xgboost as xgb
import lightgbm as lgb

def predict_with_best_ntree(bst, dmat):
    nt = getattr(bst, "best_ntree_limit", None) or
    (getattr(bst, "best_iteration", None) + 1 if getattr(bst,
    "best_iteration", None) is not None else None)
    try:
        return bst.predict(dmat, iteration_range=(0, nt)) if
nt is not None else bst.predict(dmat)
    except TypeError:
        try: return bst.predict(dmat, ntree_limit=nt)
        except TypeError: return bst.predict(dmat)

def train_regressor_ensemble(X, y, random_state=42):
    X_tr, X_te, y_tr, y_te = train_test_split(X, y,
    test_size=0.2, random_state=random_state)
    X_tr, X_val, y_tr, y_val = train_test_split(X_tr, y_tr,
    test_size=0.2, random_state=random_state)

    # XGBoost (train on log1p)
    dtrain = xgb.DMatrix(X_tr,
    label=np.log1p(np.maximum(y_tr,0)))
    dval = xgb.DMatrix(X_val,
    label=np.log1p(np.maximum(y_val,0)))
    dtest = xgb.DMatrix(X_te)

    bst =
xgb.train({'objective':'reg:squarederror','learning_rate':0.
05,'seed':random_state},
        dtrain, num_boost_round=300,
    evals=[(dtrain,'train'),(dval,'val')],
    early_stopping_rounds=20, verbose_eval=False)
```

```

# LightGBM
lgbm = lgb.LGBMRegressor(learning_rate=0.05,
n_estimators=300, random_state=random_state)
lgbm.fit(X_tr,      np.log1p(np.maximum(y_tr,0)),
eval_set=[(X_val,   np.log1p(np.maximum(y_val,0)))],
early_stopping_rounds=20, verbose=False)

# Ensemble preds (expm1 to return to original scale)
p_xgb = predict_with_best_ntree(bst, dtest)
p_lgb = lgbm.predict(X_te,
num_iteration=getattr(lgbm,'best_iteration_',None))
p = np.expm1((p_xgb + p_lgb) / 2.0)

return {'y_true': y_te, 'y_pred': p,
        'metrics': {'rmse':
mean_squared_error(y_te,p,squared=False),
                    'mae': mean_absolute_error(y_te,p),
                    'r2': r2_score(y_te,p)}}}

```

2. Classifier(XGBoost + LightGBM ensemble)

```

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score,
roc_auc_score
import xgboost as xgb
import lightgbm as lgb

def predict_with_best_ntree(bst, dmat):
    nt = getattr(bst, "best_ntree_limit", None) or
(getattr(bst, "best_iteration", None) + 1 if getattr(bst,
"best_iteration", None) is not None else None)
    try:
        return bst.predict(dmat, iteration_range=(0, nt)) if
nt is not None else bst.predict(dmat)
    except TypeError:
        try: return bst.predict(dmat, ntree_limit=nt)
        except TypeError: return bst.predict(dmat)

def train_classifier_ensemble(X, y, random_state=42):
    X_tr, X_te, y_tr, y_te = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=random_state)
    X_tr, X_val, y_tr, y_val = train_test_split(X_tr, y_tr,

```

```
test_size=0.2,                                stratify=y_tr,
random_state=random_state)
```

```
# XGBoost classifier
dtrain = xgb.DMatrix(X_tr, label=y_tr); dval =
xgb.DMatrix(X_val, label=y_val); dtest =
xgb.DMatrix(X_te)
bst =
xgb.train({'objective':'binary:logistic','eval_metric':'auc',
seed':random_state},
          dtrain, num_boost_round=300,
evals=[(dtrain,'train'),(dval,'val')],
early_stopping_rounds=20, verbose_eval=False)
```

APPENDIX B – SCREENSHOTS

The screenshot shows a Streamlit application titled "A4M — Δf & ΔG : predict formation energy and thermodynamic stability". The interface includes a sidebar with controls for uploading a train.csv file (213.5KB) and a "Run pipeline" button. The main area displays a dataset preview table with 5 rows and 10 columns. Below the table, it shows the dataset dimensions: 2400 rows, 14 columns, and 0 missing values. A green bar at the bottom indicates "Derived features, shape: (2394, 29)".

Dataset preview

	id	spacegroup	number_of_total_atoms	percent_atom_al	percent_atom_ga	percent_atom_in	lattice_vector_1_ang	lattice_vector_2_ang	lattice_vector_3_an
0	1	33	80	0.625	0.375	0	9.9523	8.5513	9.177
1	2	194	80	0.625	0.375	0	6.184	6.1838	23.628
2	3	227	40	0.8125	0.1875	0	9.751	5.6595	13.96
3	4	167	30	0.75	0	0.25	5.0036	5.0034	13.531
4	5	194	80	0	0.625	0.375	6.6614	6.6612	24.581

Rows: 2400 Cols: 14 Missing: 0

Derived features, shape: (2394, 29)

Deployed at :

<https://bhuj4eev5qfyjua6pwt3qr.streamlit.app/>